PolicyWitness architecture

This tour follows one run through the internals and across their boundaries. Read it after the README's The Experiment section and before the directory READMEs. Each section links to the relevant field reference, contract, module list, test map or build guide.

The 4 figures are generated from architecture.json by generate_architecture.py. Every node and edge in a figure has an id, and the table beside the figure names, for that id, the source symbol that implements it and the test or rule the manifest names. The generator verifies symbol presence and test definition. Follow a node or edge's id to inspect its cited implementation and checks. The hand-drawn run timeline below cites its sources the same way. Behavior that falls short of a stated promise is written where the promise is stated, in a paragraph that opens with "Known gap", and indexed in the last section.

Why these processes exist

Two facts about the macOS sandbox shape everything. `sandbox_check` answers for a live process, and `sandbox_apply` is one-way: a process that has applied a profile keeps it for the rest of its lifetime. A specimen therefore needs a fresh process that applies the policy and stays alive long enough to be queried and to attempt things. PolicyWitness spawns one fresh worker per sandboxed process the specimen names: one for the single-worker form, one per actor for the actor form. Each applies its own policy once (an unapplied actor applies none) and is queried from outside.

The reply path must stay outside the policy under test. A `(deny default)` profile can deny the worker's own writes and even its exit. The worker therefore publishes into a shared region the host mapped and pre-touched before the spawn. The unsandboxed host assembles the reply. Published attempt, native-call and progress records describe the worker's observations; the host separately observes process exit and cleanup. The disposition record keeps those sources distinct.

The host is an XPC service that launchd starts for one specimen and that exits shortly after replying, giving each run fresh host state. The controller is a separate process so that an envelope is printed whatever the host does, including never answering. The controller is Rust and `NSXPCConnection` is an Objective-C API, so a small Swift client owns the connection (why the launcher shells out).

The client acquires a connection-bound reservation before submitting specimen bytes. It may retry acquisition within one absolute deadline; a lost acquisition reply cannot have executed the specimen. Immediately before execution it records submission as invoked. No later reply loss, timeout or refusal permits replay. An unused reservation retires on expiry or connection loss; a consumed one belongs to its execution owner through reply and retirement. The user guide describes the capture fields and the distinction between invocation, host reply and effect evidence.

Known gap: the terminal reply flush delay is best effort. A completed specimen can lose its reply if the host or connection disappears; no result retrieval or safe execution replay is available.

The validator is its own process so that `sandbox_check` runs unsandboxed against the worker's PID while requested actions are held after setup. `sbpl-check` supplies an independent compile diagnostic in the controller's context when an admitted client capture ends in `xpc_error`. The held whole request retains its fallback diagnostic. For actors the controller additionally selects each object-valued actor policy and associates the helper result with that actor; this is not evidence of application. Fallback can precede host admission, so the admitted actor-count bound does not bound the number of helper invocations. Per-helper retention also does not bound their cumulative envelope or process memory; LIMITS owns those bounds.

The issuer serves the declaration form: native issuance occurs before worker creation under ordinary authority, outside the specimen policy. It receives no worker entitlement plist. The host prepares the target and observes issuance and cleanup without calling sandbox APIs. A bounded transient channel carries the token to the worker; only references and publication facts enter retained evidence. The worker consumes and releases in its existing phase sequence.

The actor form needs no issuer: an actor issues inside its own process, after its own setup, and the host moves the token between actors through fixed custody buffers and per-actor authority regions it maps before launch. The host still calls no sandbox API; it copies bytes, compares them and erases them. One actor runs requested actions at a time; parked workers still service deliveries. The handoff sequence explains completion and failure gates. No actor is designated the specimen's singular worker in the reply.

Only the worker (compile/apply, consume/release and, in an actor, file issuance), the issuer (file issuance for the declaration form), the validator (`sandbox_check`) and `sbpl-check` (compile only) call native sandbox APIs; the topology table's Native sandbox API column records these roles. Source and artifact checks enforce the host's separation through source drift and artifact inspection.

Process topology

This is an inventory of processes and channels, not a complete protocol trace. The worker node stands for each actor worker; channel arrows aggregate reciprocal traffic (notably T8 and T16–T17). The sequences below supply the order. Who starts whom, over what channel, with what lifetime. Locations are given at the granularity of "app top level" and "inside the XPC bundle"; the exact bundle paths are the contract in the README's Build and bundle. The C worker, validator and issuer sit inside the XPC bundle so that the built-in runner and a BYOXPC copy each resolve their own helpers relative to their own bundle.

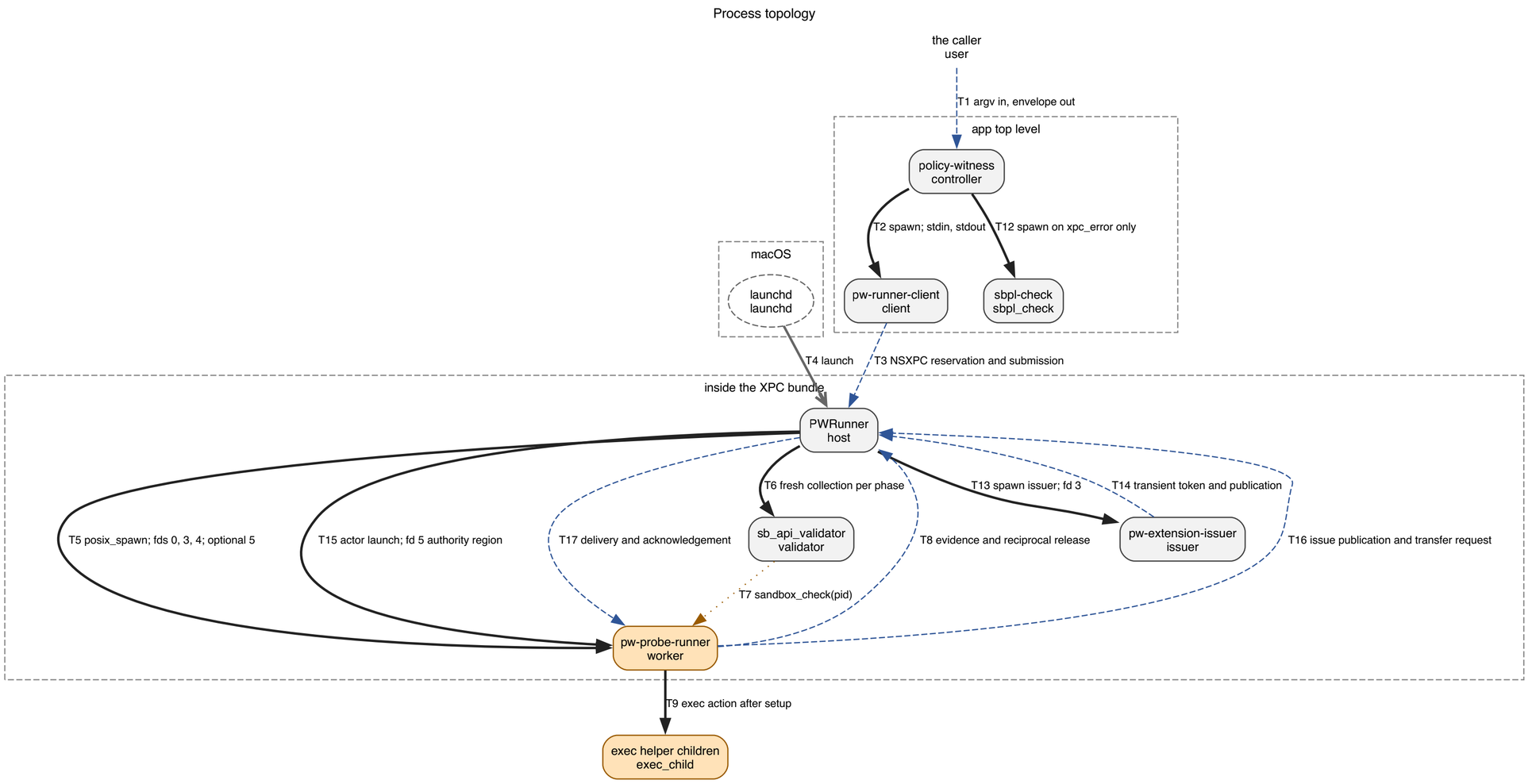


Figure: process topology. Generated from architecture.json by generate_architecture.py; dot source in architecture-topology.dot. The ids in the figure are the ids in the tables below, and each row cites the source symbol that implements it and the test or rule the manifest names for it. Symbol presence and test definition are verified; whether a test asserts the row is not verified.

Process topology: 10 nodes and 15 edges, with symbol presence and test definition verified; whether a test asserts the row is not verified

Process topology nodes

Id	Node	Kind	Where	Lifetime	Language	Started by	Sandboxed	Native sandbox API	Sources	Checks	Limits
user	the caller	person	a shell or a harness	owns the request file and reads the envelope					`pub fn run`	`check_cli_surface_agreement`	
controller	policy-witness	executable	app top level	one run	Rust	the caller	no	none	`fn main`; `cmd_run`	`consumer_controls`; `inventory`	
client	pw-runner-client	executable	app top level	one helper per request; effect-free reservation attempts and the single execution invocation share one absolute deadline	Swift	the controller	no	none	`PWRunnerClientSession`; `readToEnd`; `NSXPCConnection`	`path_wire_fixtures_survive_capture_serialization_and_independent_consumer`; `client_output_control`; `runClientSessionControls`	

Id	Node	Kind	Where	Lifetime	Language	Started by	Sandboxed	Native sandbox API	Sources	Checks	Limits
host	PWRunner	executable	inside the XPC bundle	one reservation and at most one specimen; unused reservation expiry or abandonment retires the host; terminal reply flush delay 50 milliseconds is best effort	Swift	launchd: XPC service lookup, or a Mach service for BYOXPC	no	none, by the invariance rule; actor-form token custody copies, compares and erases bytes without a sandbox call	`NSXPCListener.service()`; `runReservedSpecimen`; `hostExitDelayMs`; `AuthorityCustody`	`inspect`; `check_host_invariance`; `runServiceAdmissionTests`; `single_use`; `runActorReplyTests`	`host_exit_delay`
worker	pw-probe-runner	sandboxed	inside the XPC bundle	one specimen in the single-worker form, or one actor of an actor-form specimen; one application (none for an unapplied actor) and all of its ordered visits; parked between visits; terminal publication exits without host permission	C	the host	applies its supplied policy once; an unapplied actor declares setup without application	sandbox_compile_string, params, sandbox_apply; sandbox_extension_consume and sandbox_extension_release; sandbox_extension_issue_file in an actor	`sandbox_compile_string`; `sandbox_apply`; `publish_done_and_exit`; `attempt_extension_issue`	`test_production`; `happy_default_allow`; `runPhaseProtocolTests`; `run_targets`; `run_case`; `check_row`; `terminal_exit_without_request`; `runActorAuthorityTests`	
exec_child	exec helper children	sandboxed	the caller's path	the worker bounds collection and leader reap; ordinary completion and worker death can leave same-group descendants alive; escaped descendants are not contained	caller-supplied	the worker	inherit the worker's sandbox	none of PolicyWitness's	`attempt_exec_spawn`; `exec_resources_acquire`	`timeout_preserves_output_and_continues`; `contaminated_worker`; `closed_streams_leader_exit`; `worker_dies_during_exec`; `main`	
validator	sb_api_validator	executable	inside the XPC bundle	one phase collection	C	the host at each eligible phase arrival	no	sandbox_check	`sandbox_check`; `SANDBOX_CHECK_NO_REPORT`; `runValidator`	`predictions_do_not_report`; `bothBinariesExist`; `runPhaseProtocolTests`	
sbpl_check	sbpl-check	executable	app top level	one compile per helper process; the controller invokes a helper for the held request and one for each selected actor policy, potentially before host admission	Rust	the controller, only for an admitted client terminal xpc_error	no	sandbox_compile_string and params; never applies	`sandbox_compile_string`; `run_policy_check`	`fallback_compilation_is_requested_only_for_an_admitted_xpc_error`	
launchd	launchd	os	system		OS		no	none	`NSXPCListener(machServiceName:)`; `MachServices`	`pwrunner_survives_mach_service_launch`; `runner_install`	
issuer	pw-extension-issuer	executable	inside the XPC bundle	one declaration before worker creation; confirmed clean reap required for delivery	C	the host	outside the specimen policy; ordinary process authority	sandbox_extension_issue_file	`sandbox_extension_issue_file`; `ExtensionSession`	`run_targets`; `run_case`; `runExtensionEvidenceTests`; `check_row`	

Process topology edges

Id	From	To	Kind	Edge	Sources	Checks	Limits
T1	user	controller	channel	argv in; one JSON envelope on stdout	`pub fn run`; `print_envelope`	`check_cli_surface_agreement`; `envelope_shape_golden_agrees_with_the_manifest`	
T2	controller	client	spawn	spawn once; held request on stdin, read to EOF; versioned client capture on stdout	`run_pw_runner_client`; `readToEnd`	`path_wire_fixtures_survive_capture_serialization_and_independent_consumer`; `client_output_control`	
T3	client	host	channel	effect-free NSXPC reservation, then one runReservedSpecimen call and its reply	`NSXPCConnection`; `runReservedSpecimen`; `authorizedCaller`	`happy_default_allow`; `accepted_input_contract`	
T4	launchd	host	launch	start on XPC lookup, or as a Mach service (BYOXPC)	`NSXPCListener.service()`; `NSXPCListener(machServiceName:)`; `launchctl_bootstrap`	`pwrunner_survives_mach_service_launch`; `runner_install`	
T5	host	worker	spawn	posix_spawn; fd 0 policy pipe, fd 3 shared region, fd 4 ready pipe; optional fd 5 transient token channel	`posix_spawn_file_actions_adddup2`; `F_SETNOSIGPIPE`; `writePolicy`; `PW_SHM_ABI_MAGIC`; `PW_FAILURE_SOURCE_LENGTH`	`c_swift_layout_agreement`; `test_production`; `runPolicyTransferTests`; `run_case`; `check_row`; `policy_truncated_prefix`	

Id	From	To	Kind	Edge	Sources	Checks	Limits
T6	host	validator	spawn	fresh posix_spawn --batch per eligible phase; bundle-local default or explicit override; bounded NDJSON	`runValidator`; `decodeValidatorFrames`; `defaultValidatorExecutablePath`; `runReservedSpecimen`	`predictions_do_not_report`; `transcript_controls`; `runPhaseProtocolTests`; `runValidatorSelectionTests`; `runOrderingTests`	
T7	validator	worker	native	sandbox_check against the worker PID, with SANDBOX_CHECK_NO_REPORT	`sandbox_check`; `SANDBOX_CHECK_NO_REPORT`	`predictions_do_not_report`; `independent_worker_and_host_queries`	
T8	worker	host	channel	shared-region worker arrivals, acknowledgements, completions and native observations; reciprocal host numbered releases hold requested actions	`wait_for_proceed`; `run_attempt`; `CWorkerStepResult`; `proceedOffset`	`signal_while_waiting`; `order_barrier_mutations`; `runPhaseProtocolTests`; `run_case`; `check_row`; `proceed_precedence`	
T9	worker	exec_child	spawn	posix_spawn inside an exec action after setup (after apply when supplied); stdout/stderr pipes into bounded slot buffers; excluded for authority-participating actors	`attempt_exec_spawn`; `exec_resources_acquire`	`timeout_preserves_output_and_continues`; `test_production`	
T12	controller	sbpl_check	spawn	spawn only for an admitted client terminal xpc_error; held-request diagnostic plus independently associated actor-policy diagnostics; host request admission may not have occurred	`fallback_policy_check`; `run_policy_check`	`fallback_compilation_is_requested_only_for_an_admitted_xpc_error`; `refused_reply_versions_request_no_fallback_compilation`	
T13	host	issuer	spawn	fresh ordinary-authority issuer; fd 3 token mapping, stdio /dev/null, empty environment	`posix_spawn`; `main`	`run_targets`; `run_case`; `runExtensionEvidenceTests`; `check_row`	`extension_issuer_wait`
T14	issuer	host	channel	native issuance publication and bounded transient token bytes; host owns cleanup and delivery eligibility	`token_length`; `deliveryDescriptor`	`run_case`; `runExtensionEvidenceTests`; `check_row`	`extension_token_storage`
T15	host	worker	spawn	actor form: posix_spawn with an empty environment, stdin policy pipe, null stdout/stderr and CLOEXEC_DEFAULT; fd 5 the actor's own authority region; activation ordinal and shared lifetime cutoff in the evidence header	`posix_spawn_file_actions_addopen`; `launchAndSetUp`; `prepare_authority`	`runActorSchedulerTests`; `runActorAuthorityTests`; `activation_named_ordinal`	`actor_lifetime_cutoff`; `actor_setup_allowance`
T16	worker	host	channel	one-shot outbound publication per admitted issue and a distinct request/answer cell per transfer; the host owns published outbound bytes until copying or rejection and erasure	`attempt_extension_transfer`; `pw_authority_transfer_t`; `serviceOutbound`	`runActorAuthorityTests`; `c_swift_layout_agreement`	`actor_exchange_allowance`
T17	host	worker	channel	delivery into the recipient's assigned inbound slot; the parked recipient validates framing and its preassigned source/action/delivery before acknowledging without consuming	`serviceTransfer`; `service_inbound_once`; `pw_authority_slot_problem`	`runActorAuthorityTests`; `runActorReplyTests`	

Notes:

- `host`: All authorized connections share one locked reservation slot. Only the connection holding its grant can consume it, before specimen decoding. Stale expiry or disconnect cannot retire an executing owner; client loss after submission never permits replay.
- `T5`: Policy delivery uses nonblocking writes under one absolute monotonic deadline (the worker_policy_transfer limit) beginning after spawn. Expiry closes input and enters cleanup before readiness or sentinel polling. The pipe carries no framing: the host writes the submission's byte count into the shared header before spawn, and the worker at EOF publishes the count it read and refuses a different length before compiling.
- `T13`: Declaration form only: issuer wait 5,000 milliseconds; cleanup may overrun. Missing capability refuses declaration-form extension specimens.
- `T14`: Token storage 16,384 bytes; raw bytes never enter reply fields.
- `T15`: One region per actor, mapped by the host before launch and by that actor before application; a validator or another actor never maps it. Setup allowance 10,000 milliseconds; lifetime cutoff 180,000 milliseconds.
- `T16`: Exchange allowance 5,000 milliseconds on each side; expiry is a transport status, never a native result. A later operation cannot reuse the expired operation's payload or answer cell. Host answer and completed source action are separate observations; a source timeout can still complete its phase.
- `T17`: Recipient framing acknowledgement precedes host byte equality and usable/failed settlement, then the source answer. Consumption requires committed, accepted, usable settlement. Private custody erases at last declared transfer use, including failed delivery; shared teardown erasure requires a reaped or never-launched owner.

Every node and edge above cites at least one test or rule.

One run in time

The host closes query collection before releasing the actions of each phase. `comparison.order: "query_first"` requires that phase's eligible native record and observed collection, release, acknowledgement and ownership chain (the record). A simple `probe_plan` becomes one phase named `main`. Ordinary and declaration requests use CWorker.swift: one worker, one policy application and all its phases. Public actor requests instead reach `runActors`. ActorScheduler.swift walks their serial schedule, activating the scheduled actor's worker for each visit while others remain parked. Each actor keeps its own fresh process across its visits. Each actor applies its policy once or declares unapplied setup. Both drivers use ActorSession.swift for launch, policy transfer, readiness, polling and cleanup.

With a declaration, admission also checks references, lifecycle caps and exec exclusion. The host then resolves the submitted target and records metadata, launches its bundle-local issuer, and waits for its native result and cleanup. Only complete bounded issuance and confirmed clean reaping permit delivery. Preparation failure and other issuance failures remain observations; missing helper/native issuance capability produces a named refusal before the worker. The submitted query and attempt targets retain their original values.

```
host                                    issuer (before worker creation)
----                                    -------------------------------
resolve target; record metadata
create transient mapping; spawn --------> map fd 3; close fd; issue file token
                                          publish native result and length
                                          erase private token; exit
wait; terminate if needed; reap
validate publication and token bound
pass eligible channel as worker fd 5
```

The single-worker sequence, after any declaration issuance, is:

```
controller / client          host                       worker               validator
-------------------          ----                       ------               ---------
read; gate; select;
resolve augments;
serialize held request
stdin to EOF; XPC ----------> authorize; decode; admit
                             prepare shared region
                             spawn; policy pipe --------> compile; ready byte
                                                          apply once
                             <---------------------------- phase arrival
                             check identity + ownership
                             plan this phase afresh
                             spawn eligible collection ------------------------> sandbox_check
                             <------------------------------------------------- bounded NDJSON
                             collection closes
                             recheck phase + ownership
                             numbered release ----------> acknowledge ordinal
                                                          ordinary/native actions in order
                                                          publish completion
                             <--------------------------- next phase arrival
                             repeat collection/release    repeat phases
                                                          erase token; done; exit
                             exit request (cancellation/cleanup)
                             grace; kill if needed; reap
                             final snapshot; disposition;
                             phase/native evidence; comparisons; erase token mapping
reply <--------------------- encode; schedule host exit
admit reply version;
complete execution;
optional logs;
print envelope
```

Collection can close with unavailable predictions or incomplete validator cleanup. Release still requires a live, correctly waiting worker. A failed phase wait, invalid release ordinal or loss of worker ownership prevents later actions while preserving earlier publications. A boundary publication records progress; each action publishes its result separately. Native pre-call markers and completed result publication are distinct. A call without a published result remains unresolved even if an independent test knows it returned. A consume/release action before an ordinary action in the same phase can change what that action encounters after its query was collected. Read phase progress as the observed sequence of collections and actions. The worker exits after terminal publication without waiting for host permission. The host's mapping retains its publications. An acknowledgement acquired after confirmed reap can establish the earlier phase interval while ownership remained unbroken, including when fatal termination followed acknowledgement without `done`. The run retains its independently observed abnormal disposition.

Actor handoff and schedule advancement

After setup, each scheduled visit follows activation → arrival → eligible query collection → numbered release → worker acknowledgement → ordered actions → phase completion. Empty visits still have these boundaries. While parked, recipients service transport apparatus without running requested native actions. An issue and transfer within the active source's actions proceed as follows:

1. The source issues a token, copies bounded bytes to its one-shot outbound slot, erases/frees the native allocation and commits the slot. The host validates and copies to private custody, erases the outbound bytes, then answers accepted or rejected custody. Published bytes cannot be reused by the source even if its wait expires.
2. A transfer publishes its own request cell. The host copies custody into the assigned recipient slot. The recipient validates its preassigned framing and acknowledges while parked; a self-transfer services inbound delivery inside the source's exchange wait. Neither operation consumes the token.
3. The host compares accepted recipient bytes with custody, settles equal bytes as usable, then answers the source. Rejected or incomplete exchanges settle as failed. Only committed, accepted, usable settlement permits later consume.
4. The source publishes the answer it observed, or its own timeout/fault, in its completed action. Host source acknowledgement requires a completed action matching the host answer; a late answer cannot rewrite a source timeout.
5. The worker completes the phase after its actions return. The host services custody before checking completion and can then advance the schedule. Completion does not require every transfer to succeed: a timed-out exchange can complete with failure. Deferring recipient acknowledgement until its activation would therefore delay the earlier source's synchronous exchange.

A failed visit requires stopping and reaping its actor before further activation; unresolved termination cancels remaining activation, and timeout or ownership failures can cancel even after reap. Shared authority erasure requires a reaped or never-launched owner; private host custody is erased separately. The worker resource account gives the lifetimes. This sequence follows `walkSchedule`, `serviceOutbound`, `serviceTransfer` and `recordSourceAcknowledgements`, and the worker's `wait_exchange` and `service_inbound_once`. Actor authority controls and handoff mutation controls cover copying, parked acknowledgement, settlement and quiescence dependencies.

Implementation and checking routes

Where each stage is pinned:

- **Controller admission and selection.** `run` in run_flow.rs: the manifest is loaded once, then `parse_request`, `validate_request_version`, selector parsing, `resolve_runner_target_with_registry`, `resolve_augments`, the dossier, and `strip_runner_selector` before the held bytes are serialized. Checked by the `orchestration` tests in the same file.
- **Extension preparation and issuer lifetime.** `ExtensionSession` records target resolution and metadata before a fresh native issuer, admits bounded publication only after confirmed clean reap, and clears the mapping. `run_targets` checks placement, symlinks, preparation failure and replacement independently; `run_case` drives native interruption, malformed transport and token confinement. `runExtensionEvidenceTests` inspects actual mapping erasure and withholds delivery after an unconfirmed reap.
- **Caller authorization.** `authorizedCaller` runs when the connection is accepted, before any request is read.
- **Host admission.** `runSpecimen` decodes with closed keys, then `admissionFailure(for:)` (capacity), `requestMeaningFailure` (meaning) and `computePolicyHash`, in that order; a refusal replies without spawning anything. Checked by `runner_outcome_bad_request` and `failure_boundaries`.
- **Query planning.** `planValidatorQueries` runs anew at each observed phase arrival, after all prior phase actions have completed. It drops excluded operation/filter pairs and paths that do not resolve on the host then. `path_unresolved_at_planning` supplies an explicit unavailable prediction without suppressing the paired attempt. Checked by `PhaseProtocolTests`, query admission tests and the planner rule in `source_drift`.
- **Spawn and apply.** The driver's header comment in CWorker.swift from the region setup through the ready byte, and the worker's from mapping the region through apply. The ready byte has its own window (`readyByteTimeoutMs`); `runner_ready_byte_resilience` checks it.
- **Compiled-object receipt.** When the request opts in, the worker copies the compiled object into its capture region before applying it (`pw_capture_profile`). The host admits that copy only after successful application, worker completion, PID, nonce, input identity and payload integrity checks (`decodeProfileCapture`); the guide defines the receipt's provenance and reading rules. In actor form the receipt belongs to the opted-in apply actor and is admitted using that actor's identity and completion; unapplied setup supplies no receipt. Checked by `CompiledProfileCaptureTests`.
- **The barrier.** The `phaseBoundary` callback collects against the worker's PID. The driver observes arrival and checks child ownership before collection and again before release. Its ordinal store cannot reuse an earlier release; immutable phase acknowledgement records preserve earlier observations. `PWRunnerOrdering` is assembled independently per phase. `PhaseProtocolTests`, `OrderingTests` and the opt-in order-barrier mutation control check the protocol.
- **Attempts and publication.** `run_attempt` writes each slot and releases `completed`; an exec attempt acquires its pipes and spawn handles after apply and releases them before its slot completes. Checked by `runner_c_worker_harness` and `runner_exec_lifecycle`.
- **Exit, grace and kill.** The worker harness's `terminal_exit_without_request` and `terminal_erasure` controls cover autonomous exit and erasure before publication; `runner_outcome_runner_timeout/host_kills_hung_worker` checks the kill path. `runCWorkerLifecycleTests` covers acknowledgement recovery after confirmed reap without restoring a broken ownership chain.
- **Reply and envelope.** `reply_version` admits the reply or retains it unread; `fallback_policy_check` runs only for an admitted `xpc_error`; `complete_execution` projects the disposition; `execution_envelope` assembles the execution records. Checked by the Rust unit tests named in the boundary tables below.

The client's timeout ends the controller's wait and yields a synthetic `xpc_timeout` reply. Host work continues independently. The host's cleanup can fail: `terminate` makes one blocking wait after a successful kill request, and when no reap is confirmed the collection basis reads `execution_may_continue` rather than `after_confirmed_reap`. The failure evidence contract records reply delivery, the end of observation and the end of execution separately. There is no global lifecycle deadline.

The worker tour owns the exec-descendant limitation and explains the resources and observations retained during exec cleanup.

Each budget covers a specific stage. In the declaration form the issuer's monotonic wait, `extension_issuer_wait`, begins after issuer spawn and precedes worker creation. Target preparation and spawn are outside that wait; issuer cleanup can overrun it. The client reply window includes this work, while worker and phase collection budgets have not started. Policy delivery has its own absolute monotonic deadline, `worker_policy_transfer`, started immediately after the spawn; the ready window, `worker_ready_wait`, covers only the wait for the ready byte after the policy has been written. In the single-worker driver the sentinel deadline, `worker_sentinel_wait`, covers application through `done`, less every phase collection interval, because callbacks run synchronously between polls; the validator I/O deadline, `validator_io_wait`, covers one collection, capped by the remaining cumulative `phase_collection_wait` allowance for planning and collection across all phases. Cleanup time is charged to that allowance but can overrun it; exhausted later phases retain unavailable predictions. The proceed budget, `worker_proceed_wait`, covers each worker phase's wait for its exact release ordinal; the grace timer, `worker_exit_grace`, covers exit request to kill; the exec deadline and attempt budget, `exec_child_wait` and `exec_attempt_budget`, cover the worker's waits on exec children, per step and per worker plan, excluding acknowledged phase release waits and, for actors, accepted parked activation waits. The worker's release wait and exec allowances count continuous Mach time, which includes system sleep, in one validated tick domain; host waits keep their own clock, and no worker time is compared with them. The ready and sentinel budgets are elapsed monotonic allowances: each sleep is bounded by what remains, polling costs consume it, the sentinel allowance excludes measured phase collection intervals, and expiry starts cleanup rather than promising a total response time. These `HostWaitClock` waits fall back to remaining nominal sleeps on a clock fault. Their values and checks are in LIMITS.md.

Actors instead have one setup allowance starting before authority-region preparation, one execution pool for the whole schedule excluding synchronous collection, and one aggregate collection pool reducible by `validator_total_timeout_ms`. Policy delivery and readiness are capped by remaining setup time. The legacy sentinel override does not change the actor execution pool. The shared continuous lifetime cutoff is sampled at checked worker waits; it does not preempt blocked native calls or bound final cleanup. These allowances are distinct from the client's reply deadline and each worker's exec budget; see execution budgets.

The host sends policy bytes through a nonblocking pipe under the delivery deadline (`writePolicy` and `MonotonicDeadline` in MonotonicDeadline.swift). Partial writes, interruptions and backpressure consume the same allowance. Expiry records `policy_transfer_timeout` separately from any write errno, closes input and enters cleanup without starting readiness or sentinel polling. Counts describe writes accepted by the pipe. The worker's account of what it read is separate: the host writes the submission's byte count into the shared header before spawn, and the worker, at EOF, publishes the count it read (`policy_read` in the reply) and refuses a different length before compiling, so a delivery cut after a complete form is never applied as a shorter policy. `PolicyTransferTests` checks a stalled reader, exact draining delivery, deadline edge cases, cleanup failures and the truncated delivery the shipped worker refuses after the host's timeout; the actual driver replies pass through controller assembly and the independent consumer. The worker harness pins the refusal, the receipt and both directions of the comparison without the host. The deadline does not bound final reap or cancel an earlier client timeout.

Evidence channels and ownership

The channels that reach the envelope are tabulated below. Each native result has one writer. The host preserves native results and adds its own observations beside them; the controller does the same with the host's.

Channel	Native result written by	Host adds	Where it lands	What it is
Prediction	the validator	an explicit `prediction_unavailable` observation for a step the planner excluded; path diagnostics after orchestration	`steps[].sandbox_check`	one `sandbox_check` verdict per predicted step
Attempt	the worker, through the shared region	the lifecycle copies; path diagnostics after orchestration	`steps[].attempt`	the attempted operation's own result, with `rc` and `errno`
Extension preparation/issuance	issuer publishes native facts	resolved target/metadata, process authority and independent cleanup; delivery and host erasure observations	`extensions[]`	target identity at preparation, native issuance and delivery observations
Native lifecycle	worker call marker and result slot	phase/reference association and lifecycle claims; for an actor, the acting actor, family, class, target and token length	`extension_events[]`	consume handle, release result or issue result when published
Transfer	none; host transport exchange	custody, copy, byte equality, recipient and source acknowledgement	`transfers[]`	one explicit exchange between actors, with no native return attached
Resources	none: a join over the actor and worker records	token, delivery and handle tables keyed by the request's references with owners and last uses	`resources`	which reference is which, who owns it and what became of it
Actor disposition	each actor's host, from the raw facts it cites	the controller's per-actor projection	`actors[].subprocess.disposition`; `data.runner_sandbox_diagnostics.actors[]`	what happened to each actor's process, answered only from the cited facts
Fresh read	worker syscalls inside the requested action after setup	phase and step association	`steps[].attempt.fresh_read`	fresh descriptor identity and bounded bytes, separate open/stat/read/close results

Channel	Native result written by	Host adds	Where it lands	What it is
Comparison	the host	nothing further	`steps[].comparison`	the attempt's classified observation, the submitted-scope relations, the order established, and the limitations
Disposition	the host, from raw facts it cites	the controller's projection	`runner_subprocess.disposition`; `data.runner_sandbox_diagnostics` (execution fields)	what happened to the worker process, answered only from the cited facts
Compiled-object receipt	the worker, before apply, when asked	admission after application, completion, PID, nonce and integrity checks	`compiled_profile`, or `actors[].compiled_profile`	a worker copy of the compiled object before apply
Dossier	the controller	nothing further	`data.specimen`	the request as submitted, augmentation and imports, host facts, runner and app provenance, binary hashes observed before invocation

The host adds path diagnostics through `enrichPathDiagnostics` in PWRunnerService.swift and synthesizes excluded-query results from planner decisions in CWorkerStepResults.swift. The failure evidence contract defines the reply's normative semantics and the comparison record's reading rules.

Boundaries

Each node below is a record that crosses from one owner to another, and each edge is the guard that lets it cross. contract.json owns the wire version numbers. The identity generator produces one host/worker source identity; the build checks its copies and refuses stale ones without rewriting them (CONTRACT.md). The records that carry a local version field of their own, each read by exactly one receiver, are the evidence manifest, the runner registry, the validator NDJSON and the compiled-object receipt, and the node table's Form column names each one's field. Every other boundary is held by a golden, a suite, or by being built and shipped together, and the edge table says which.

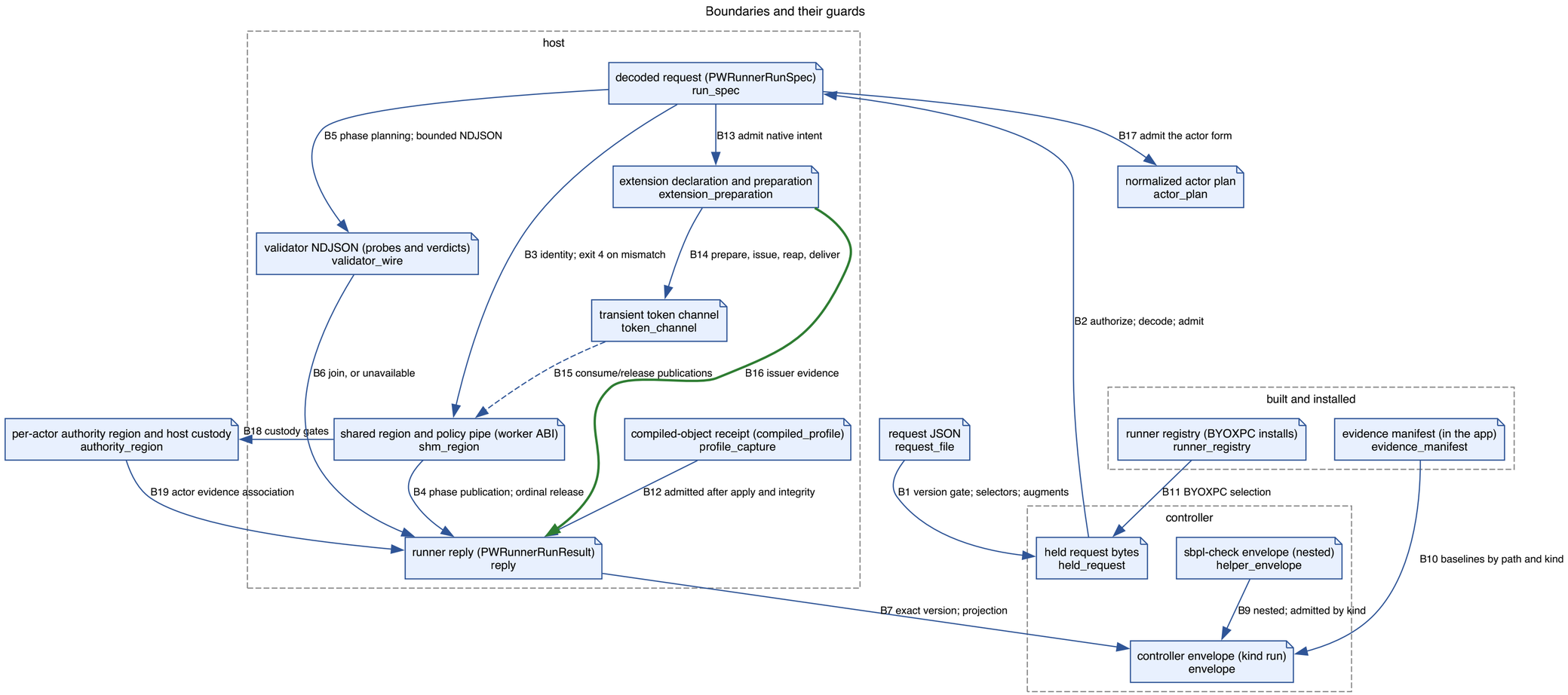


Figure: boundaries and their guards. Generated from architecture.json by generate_architecture.py; dot source in architecture-boundaries.dot. The ids in the figure are the ids in the tables below, and each row cites the source symbol that implements it and the test or rule the manifest names for it. Symbol presence and test definition are verified; whether a test asserts the row is not verified.

Boundaries and their guards: 15 nodes and 18 edges, with symbol presence and test definition verified; whether a test asserts the row is not verified

Boundaries and their guards nodes

Id	Node	Kind	Owner	Form	Sources	Checks	Limits
request_file	request JSON	record	the caller	a JSON file naming the request schema	`parse_request`	`accepted_input_contract`	
held_request	held request bytes	record	the controller	one serialization; selectors stripped, augments spliced	`strip_runner_selector`; `pub fn resolve_augments`	`invalid_request_version_stops_before_selection_augments_or_client_invocation`	
run_spec	decoded request (PWRunnerRunSpec)	record	the host	Codable with closed keys	`PWRunnerRunSpec`; `CodingKeys`	`accepted_input_contract`; `admission`	
shm_region	shared region and policy pipe (worker ABI)	record	host writes inputs, worker writes outputs	fixed layout; magic at byte 0, source identity at byte 64	`PW_SHM_ABI_MAGIC`; `PW_WORKER_ABI_IDENTITY_HEX`; `PWShmLayout`	`c_swift_layout_agreement`; `PW_SHM_`	
validator_wire	validator NDJSON (probes and verdicts)	record	host and validator, co-shipped in one bundle	bounded NDJSON; exact kind sb_api_validator_verdict and schema_version 1	`sb_api_validator_verdict`; `decodeValidatorFrames`; `sandbox_check`	`predictions_do_not_report`; `validator_overlong_request`; `runValidatorEvidenceTests`	

Id	Node	Kind	Owner	Form	Sources	Checks	Limits
reply	runner reply (PWRunnerRunResult)	record	the host	response schema; shape golden	`PWRunnerRunResult`; `PWContract`	`replyShapeDocuments`; `schema_version`	
profile_capture	compiled-object receipt (compiled_profile)	record	the opted-in worker before apply; admitted by the host for that worker or apply actor	opt-in copy of the compiled object with a nonce; CompiledProfileCapture names the worker producer and before_apply stage under its own schema_version	`pw_capture_profile`; `CompiledProfileCapture`	`runCompiledProfileCaptureTests`	
envelope	controller envelope (kind run)	record	the controller	envelope frame; shape golden	`SCHEMA_VERSION`; `render_envelope`; `complete_execution`	`envelope_shape_golden_agrees_with_the_manifest`	
helper_envelope	sbpl-check envelope (nested)	record	sbpl-check	envelope frame; kind sbpl_check; independently compiled diagnostic associated with the selected policy, not application evidence	`run_policy_check`; `sandbox_compile_string`	`_validate_policy_check_reply`; `policy_check_controls`	
evidence_manifest	evidence manifest (in the app)	record	the build	hashes and entitlements of every shipped binary; its own schema_version (EVIDENCE_SCHEMA_VERSION)	`manifest.json`; `EVIDENCE_SCHEMA_VERSION`	`codesign.preflight`; `dossier_witness`	
runner_registry	runner registry (BYOXPC installs)	record	the runner commands	additive-field loader; retired kind aliases kept for recovery; its own schema_version (RUNNER_REGISTRY_SCHEMA_VERSION)	`load_registry`; `RUNNER_REGISTRY_SCHEMA_VERSION`; `alias = "machme"`	`legacy_machme_kind_deserializes_as_byoxpc`; `runner_install`	
extension_preparation	extension declaration and preparation	record	host; submitted references and paths remain distinct from observations	bounded references, realpath result and metadata before issuer launch	`extensionMeaningFailure`; `realpath`	`run_targets`; `runRequestContractTests`	
token_channel	transient token channel	record	host owns mapping; issuer writes token/publication; worker consumes and erases	separate bounded shared mapping; inherited descriptors closed after mapping; never retained JSON	`PWExtensionLayout`; `extension_channel`	`c_swift_layout_agreement`; `run_case`; `runExtensionEvidenceTests`; `check_row`	
actor_plan	normalized actor plan	record	host normalization before process work; admitted public actor requests dispatch to runActors and its serial scheduler	actors, phases with actor and visit, tokens, deliveries and handles by reference with owners, producers and last uses	`normalizeActorPlan`; `ActorTranslation`	`runActorPlanTests`; `runRequestContractTests`	
authority_region	per-actor authority region and host custody	record	host creates, unlinks and maps each region; the actor maps its inherited descriptor; custody buffers are host-only	one outbound slot per admitted issue, one inbound slot per recipient delivery and one answer cell per transfer; fixed headers with family, writer, indexes, length, commit, acknowledgement and settlement words; never retained JSON	`pw_authority_slot_t`; `PWAuthorityLayout`; `TokenCustody`	`c_swift_layout_agreement`; `runActorAuthorityTests`	

Boundaries and their guards edges

Id	From	To	Kind	Edge	Sources	Checks	Limits
B1	request_file	held_request	guard	request version gate; selector parse; augment resolution	`validate_request_version`; `parse_runner_selector_value`; `pub fn resolve_augments`	`invalid_request_version_stops_before_selection_augments_or_client_invocation`; `accepted_input_contract`	
B2	held_request	run_spec	guard	caller authorization; closed-key decode; capacity, then meaning; policy hash	`authorizedCaller`; `CodingKeys`; `admissionFailure(for`; `requestMeaningFailure`; `computePolicyHash`	`admission`; `runEnvelopeInvariantTests`	
B3	run_spec	shm_region	guard	host writes identity, counts, phase partitions, the expected policy length and slots; worker refuses mismatched identity, invalid partition, an expectation the source buffer cannot hold, or an unusable timebase or allowance before any policy byte	`abiIdentityHex`; `PW_WORKER_ABI_IDENTITY`; `return 4`; `SOURCE_DIRS`; `PW_FAILURE_LENGTH_EXPECTATION`; `pw_timing_setup`	`c_swift_layout_agreement`; `independent_worker_and_host_queries`; `test_production`; `runPhaseProtocolTests`; `policy_expectation_out_of_range_refused`; `clock_arithmetic`	
B4	shm_region	reply	guard	per-phase release/acquire publication; exact ordinal before actions after a validated continuous-time decision; disposition from raw facts; collection status agrees with validator observations; native result publication and fresh-read consistency before claims	`wait_for_proceed`; `pw_clock_sample`; `proceedOffset`; `PWRunnerOrdering`; `extensionEvidenceProblem`; `collectionEvidenceConsistent`; `publish_done_and_exit`; `observeAcknowledgement`	`signal_while_waiting`; `order_barrier_mutations`; `runPhaseProtocolTests`; `runExtensionEvidenceTests`; `phase_collection_states_gate_disposition_without_requiring_success`; `run_extension_controls`; `proceed_precedence`; `runWorkerClockTests`; `terminal_exit_without_request`; `runCWorkerLifecycleTests`	

Id	From	To	Kind	Edge	Sources	Checks	Limits
B5	run_spec	validator_wire	guard	fresh phase planning; bounded strict NDJSON decode; unique step association within the phase	`planValidatorQueries`; `decodeValidatorFrames`	`predictions_do_not_report`; `transcript_controls`; `runPhaseProtocolTests`	
B6	validator_wire	reply	guard	verdict joined only to its phase and step, or explicit unavailable prediction	`ComparisonEvidence`; `sb_api_validator_verdict`	`transcript_controls`; `happy_default_allow`; `runPhaseProtocolTests`	
B7	reply	envelope	guard	exact response version; disposition validated and projected; reply retained unread when refused	`reply_version`; `complete_execution`; `execution_diagnostics`; `validate_disposition`	`reply_versions_are_gated_exactly`; `rust_wrapper_controls`	
B9	helper_envelope	envelope	guard	nested unchanged and associated with the selected policy/actor; admitted by kind and frame version; per-helper retention is not a cumulative envelope or memory bound	`fallback_policy_check`; `_validate_policy_check_reply`	`fallback_compilation_is_requested_only_for_an_admitted_xpc_error`; `policy_check_controls`	
B10	evidence_manifest	envelope	guard	dossier compares selected binaries with manifest baselines; lookup by exact path and kind	`unique_typed_entry`; `pub struct Binaries`	`codesign.preflight`; `dossier_witness`	
B11	runner_registry	held_request	guard	selection resolves a BYOXPC target; selector fields stripped before XPC	`resolve_runner_target_with_registry`; `strip_runner_selector`; `load_registry`	`legacy_machme_kind_deserializes_as_byoxpc`; `runner_install`	
B12	profile_capture	reply	guard	admitted only after successful application, completion, PID, nonce, input identity and payload integrity checks for its producing worker/apply actor	`decodeProfileCapture`	`runCompiledProfileCaptureTests`	
B13	run_spec	extension_preparation	guard	explicit phases, declaration/action caps, reference order and exec exclusion before preparation	`extensionMeaningFailure`; `requestAdmissionFailure`	`runRequestContractTests`; `run_targets`	`extension_consumes`; `extension_declarations`; `extension_reference`; `extension_releases`; `extension_target`
B14	extension_preparation	token_channel	guard	resolved target and metadata before native issuance; clean issuer reap and bounded publication before worker delivery	`ExtensionSession`; `PW_EXTENSION_TOKEN_BYTES`	`run_targets`; `run_case`; `runExtensionEvidenceTests`; `check_row`	
B15	token_channel	shm_region	channel	worker receives transient token separately; publishes delivery and ordered consume/release facts in evidence slots	`sandbox_extension_consume`; `sandbox_extension_release`; `erase_accepted_token`	`run_case`; `run_targets`; `check_row`; `terminal_erasure`	
B16	extension_preparation	reply	writes	preparation, authority and native issuer/cleanup observations without token bytes	`PWRunnerExtensionRecord`; `record`	`runExtensionEvidenceTests`; `run_targets`	
B17	run_spec	actor_plan	guard	closed actor and sandbox objects, phases naming declared actors, single-assignment references used after their producers by their owners, and the per-actor exec exclusion	`normalizeActorPlan`; `actorAdmissionFailure`	`runActorPlanTests`; `main`	`actor_deliveries`; `actor_handles`; `actor_id`; `actor_issues`; `actor_phases`; `actor_releases`; `actors`; `extension_issue_target`
B18	shm_region	authority_region	guard	an outbound token enters custody only with the expected framing, writer, action and token indexes, and only while the writer still stands behind it; a delivery is credited only when the recipient's accepted bytes equal the custody copy	`slotFramingProblem`; `serviceTransfer`; `pw_authority_slot_problem`	`runActorAuthorityTests`	
B19	authority_region	reply	guard	every actor event names its actor and one of that actor's phases, references resolve to their owners in the resource tables, transfers carry no native result, and nothing is credited without the actor's declared or applied setup and its phase's acknowledged release	`actorExtensionEvidenceProblem`; `actor_extension_issues`; `validate_actor_extensions`	`runActorReplyTests`; `runContractVersionTests`	

Notes:

- `B4`: Terminal publication erases accepted token storage, releases done and exits. The host retains its mapping and reconciles acknowledgement after confirmed reap only while ownership through acknowledgement remains established, including abnormal exit without done. Poll stop, deadline and disposition remain independent.
- `B13`: Declaration count 1 item; consume count 1 item; release count 1 item; references 63 UTF-8 bytes; target 1,023 UTF-8 bytes.

- `B15`: Accepted token storage is erased on ordinary departures and before done; erasure does not invoke native release. Fatal interruption need not reach worker cleanup. Host erasure is a separate observation.
- `B17`: Actors 8 items; phases 16 items; issues 32 items; deliveries 64 items; handles 64 items; releases 64 items; identifiers 63 UTF-8 bytes; issue target 1,023 UTF-8 bytes.
- `B18`: Recipient acceptance is framing only; host equality and usable settlement precede native consume. Source-observed answer and phase completion are separate gates.
- `B19`: A failed exchange may still complete its action and phase; schedule advancement does not require all deliveries to succeed. No native permission or revocation follows from transport evidence.

Every node and edge above cites at least one test or rule.

The validator receiver checks an exact kind and schema marker, bounded frames and strings, and per-query byte and record allowances before association. A rejected frame retains bounded context and prior complete records. Co-shipping and the worker source identity do not authenticate a validator selected through a test seam; its observations and process disposition remain explicit.

The controller and the nested `data.policy_check.envelope` share the envelope frame. The helper receiver checks its kind and version in policy_check.rs before interpreting its outcome; rejected parsed output is retained as opaque evidence.

How the system verifies itself

The suites are listed in tests/README.md; this section names the mechanisms they are built from.

- **Goldens.** `response_shape.json` and `envelope_shape.json` under `tests/fixtures/contract/` record every key a reply or envelope can carry and its type, collected from field-complete fixtures; they are also the readers' allowlists. `abi_layout.txt` is the compiled harvest of the worker ABI's sizes and offsets. A difference writes a candidate; replacing the golden with the reviewed candidate is the acknowledgement (shape goldens).
- **Generators with marked regions.** The 8 manifest nodes of the document graph, from the wire numbers to the prose baseline, are hand-owned facts, and the identity is digested from the sources themselves. The 6 generator nodes, one of them the shared module, copy manifest facts into marked regions of documents and sources, render counts and limit values into inline spans, and resolve limit placeholders in the architecture facts; the build runs every generator's check before signing. Every check citation carries a form, and the generator verifies the definition behind a test. Prose that still states a value, a count or a citation outside these mechanisms is listed in the prose baseline, which grows only by an explicit entry and which a release cannot grow. These documentation input manifests feed build-time checks. The embedded evidence manifest is read by the controller during a run, as the run inventory describes.
- **Source-drift rules.** Mechanical checks over text that is not generated, from the host invariance rule to the harness note's copies. Each rule is a citation of the drift node in the document graph, and a test holds that list equal to the rules the script runs.
- **The consumer.** consumer.py gates an envelope on exact versions, validates every step against its raw channel fields through the goldens, and selects steps by field; it is the reader the suites share and the reader an external consumer is expected to copy.
- **Request-exposed test controls.** `_test_overrides` can select deadlines, hangs, a self-signal, and replacement worker, validator or issuer executables. Supplied controls are echoed when available, subject to refusal sanitization; an echo does not prove that execution reached the selected boundary. The supported keys are the table in runner/README.md, locked to the Swift type by a drift rule, and the assertion recipe is in runner/AGENTS.md. Direct driver tests also inject process calls, and native extension fixtures instrument separately compiled helpers; these are distinct test boundaries.
- **The C harness.** `runner_c_worker_harness` drives the real worker through the shared-memory ABI from a C program with no host, so the worker's publication and failure paths are pinned independently of the Swift driver, including the actor-mode activation wait, the declared unapplied setup and the worker's own policy hash.
- **The actor driver tests.** `runActorSchedulerTests` drives the internal actor scheduler against the shipped worker and validator. Process identity, each actor's policy and the quiescence of parked actors are checked through the kernel process table, validator verdicts against the live PID and the other sessions' own publications, not through the scheduler's labels.
- **The lifecycle oracle.** lifecycle_oracle.py constructs envelopes from worker records and raw facts and checks that the disposition projection says only what those facts support, with disabled unrelated fields so only supporting evidence can decide a lifecycle claim.
- **The dispatcher's artifact check.** Before any app-dependent case runs, the dispatcher verifies the bundle layout against `EXECUTABLES`, the signatures, the embedded evidence hashes and the host's undefined symbols; fixture bundles with a damaged seal or a sandbox-importing host prove the check refuses them.

The last figure is the graph those generators and rules draw over the documents: which manifest feeds which generator, which regions it writes, which copies a rule keeps equal, and where the build runs the checks.

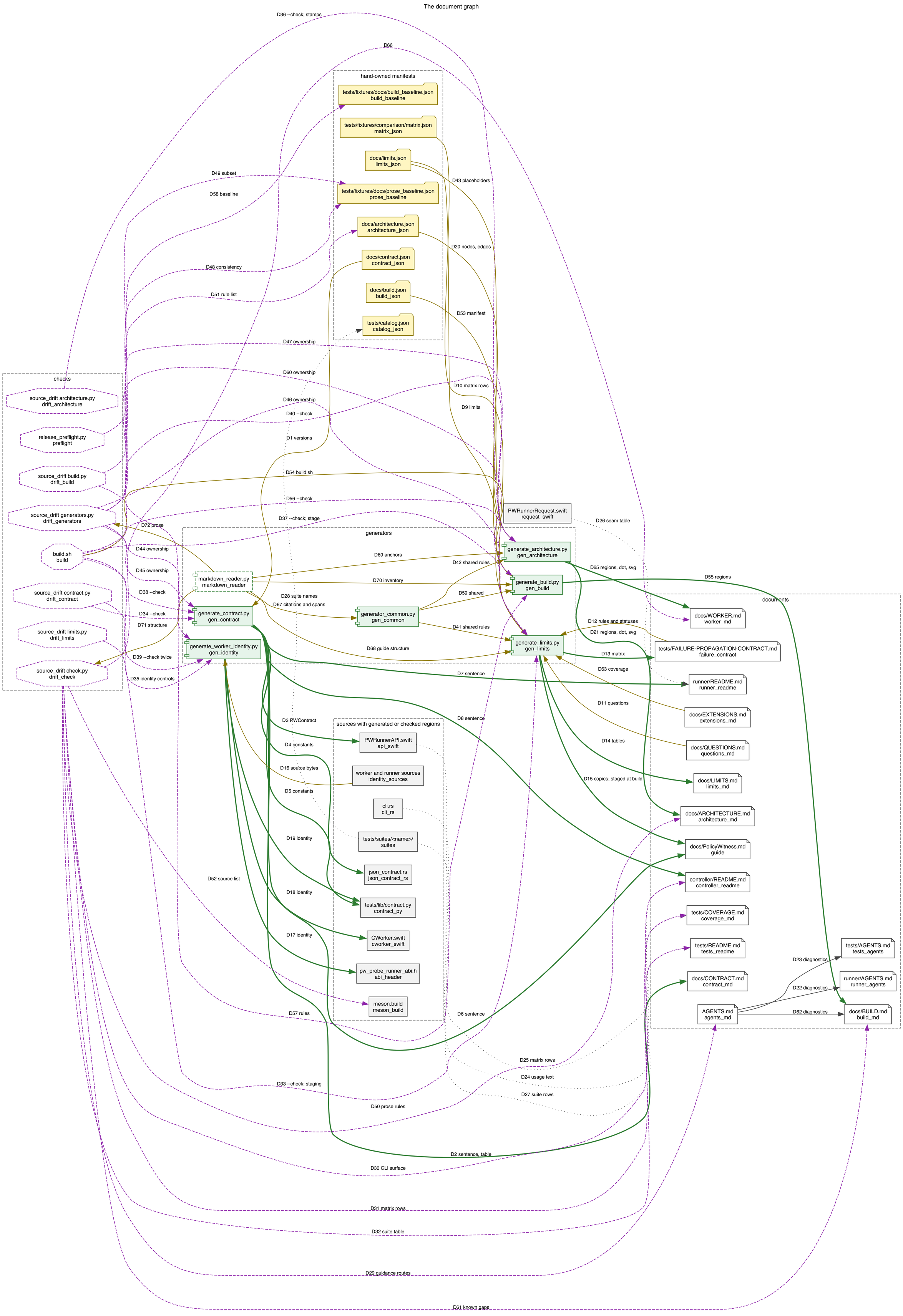

The document graph
D36 --check; stamps
D66
hand-owned manifests
tests/fixtures/docs/build_baseline.json
build_baseline
tests/fixtures/comparison/matrix.json
matrix_json
docs/limits.json
limits_json
D49 subset
D58 baseline
D43 placeholders
tests/fixtures/docs/prose_baseline.json
prose_baseline
docs/architecture.json
architecture_json
D20 nodes, edges
docs/contract.json
contract_json
D48 consistency
docs/build.json
build_json
D51 rule list
D53 manifest
tests/catalog.json
catalog_json
D47 ownership
D60 ownership
D10 matrix rows
D46 ownership
D9 limits
D40 --check
D1 versions
checks
source_drift architecture.py
drift_architecture
release_preflight.py
preflight
source_drift build.py
drift_build
D54 build.sh
D56 --check
source_drift generators.py
drift_generators
D37 --check; stage
PWRunnerRequest.swift
request_swift
D26 seam table
D72 prose
generators
build.sh
build
D44 ownership
D69 anchors
generate_architecture.py
gen_architecture
D42 shared rules
D65 regions, dot, svg
D45 ownership
markdown_reader.py
markdown_reader
D70 inventory
generate_build.py
gen_build
D55 regions
source_drift contract.py
drift_contract
D38 --check
D28 suite names
D59 shared
documents
D67 citations and spans
generate_contract.py
gen_contract
generator_common.py
gen_common
docs/WORKER.md
worker_md
D34 --check
D12 rules and statuses
source_drift limits.py
drift_limits
D71 structure
D21 regions, dot, svg
D41 shared rules
generate_limits.py
gen_limits
generate_worker_identity.py
gen_identity
D68 guide structure
D13 matrix
tests/FAILURE-PROPAGATION-CONTRACT.md
failure_contract
D39 --check twice
D63 coverage
source_drift check.py
drift_check
D35 identity controls
D7 sentence
runner/README.md
runner_readme
D11 questions
docs/EXTENSIONS.md
extensions_md
D3 PWContract
D8 sentence
sources with generated or checked regions
D14 tables
docs/QUESTIONS.md
questions_md
PWRunnerAPI.swift
api_swift
D4 constants
D15 copies; staged at build
docs/LIMITS.md
limits_md
D16 source bytes
worker and runner sources
identity_sources
D5 constants
cli.rs
cli_rs
docs/ARCHITECTURE.md
architecture_md
D19 identity
tests/suites/<name>/
suites
docs/PolicyWitness.md
guide
json_contract.rs
json_contract_rs
D52 source list
controller/README.md
controller_readme
D18 identity
tests/lib/contract.py
contract_py
tests/COVERAGE.md
coverage_md
D17 identity
CWorker.swift
cworker_swift
tests/README.md
tests_readme
tests/AGENTS.md
tests_agents
D23 diagnostics
pw_probe_runner_abi.h
abi_header
docs/CONTRACT.md
contract_md
runner/AGENTS.md
runner_agents
D22 diagnostics
meson.build
meson_build
AGENTS.md
agents_md
D62 diagnostics
docs/BUILD.md
build_md
D57 rules
D6 sentence
D25 matrix rows
D24 usage text
D27 suite rows
D33 --check; staging
D50 prose rules
D2 sentence, table
D30 CLI surface
D31 matrix rows
D32 suite table
D29 guidance routes
D61 known gaps

Figure: the document graph. Generated from architecture.json by generate_architecture.py; dot source in architecture-documents.dot. The ids in the figure are the ids in the tables below, and each row cites the source symbol that implements it and the test or rule the manifest names for it. Symbol presence and test definition are verified; whether a test asserts the row is not verified.

The document graph: 49 nodes and 71 edges, with symbol presence and test definition verified; whether a test asserts the row is not verified

The document graph nodes

Id	Node	Kind	Owns	Carries	Guards	Sources	Checks	Limits
contract_json	docs/contract.json	manifest	the request, response and envelope version numbers			`controller_envelope`	`test_every_generated_copy_is_current`	
limits_json	docs/limits.json	manifest	every runtime limit with its source and check citations			`schema_version`	`test_document_is_current`	
matrix_json	tests/fixtures/comparison/matrix.json	manifest	the comparison scenario matrix			`observation`	`test_document_is_current`	
architecture_json	docs/architecture.json	manifest	the nodes and edges of these figures, each with its citations			`MANIFEST_NAME`	`test_document_and_figures_are_current`	
catalog_json	tests/catalog.json	manifest	the registered suites and cases			`"suites"`	`check_index_vs_disk`	
gen_contract	generate_contract.py	generator	the generated contract-version regions			`TARGETS`	`test_every_generated_copy_is_current`	
gen_limits	generate_limits.py	generator	the limits tables, the scenario matrix, the guide's copied regions and the limits spans			`CONTRACT_NAME`	`test_document_is_current`	
gen_identity	generate_worker_identity.py	generator	the host/worker source identity regions			`SOURCE_DIRS`	`test_identity_is_current_relocatable_and_regeneration_is_idempotent`; `test_layout_and_both_handshake_implementations_change_identity`	
gen_architecture	generate_architecture.py	generator	these figures, their dot and SVG files, the tables beside them and the architecture spans			`MANIFEST_NAME`	`test_document_and_figures_are_current`	
contract_md	docs/CONTRACT.md	document		the version sentence and table		`BEGIN GENERATED CONTRACT VERSIONS`	`test_every_generated_copy_is_current`	
limits_md	docs/LIMITS.md	document		the limits tables and coverage		`BEGIN GENERATED LIMITS`	`test_document_is_current`	
guide	docs/PolicyWitness.md	document		copied limits, questions, reading rules and extension statuses and coverage; the version sentence; the only document that ships		`BEGIN COPIED LIMITS`; `BEGIN GENERATED CONTRACT VERSIONS`	`test_document_is_current`; `--stage-guide`	
questions_md	docs/QUESTIONS.md	document		the shared questions the guide copies		`BEGIN SHARED QUESTIONS`	`test_document_is_current`	
failure_contract	tests/FAILURE-PROPAGATION-CONTRACT.md	document		the generated scenario matrix, shared reading rules and extension statuses		`BEGIN GENERATED SCENARIO MATRIX`; `BEGIN SHARED READING RULES`; `BEGIN SHARED EXTENSION STATUSES`	`test_document_is_current`	
architecture_md	docs/ARCHITECTURE.md	document		these figures and their tables		`REGION_START`	`test_document_and_figures_are_current`	
agents_md	AGENTS.md	document		repository working rules and sandbox guidance with routes to local diagnostics		`Sandboxed automation harnesses`	`check_harness_guidance_routes`	
controller_readme	controller/README.md	document		the CLI surface block and the version sentence		`CLI surface (contract)`; `BEGIN GENERATED CONTRACT VERSIONS`	`check_cli_surface_agreement`	
runner_readme	runner/README.md	document		the test-seam table and the version sentence		`_test_overrides`; `BEGIN GENERATED CONTRACT VERSIONS`	`check_test_overrides_table_agreement`	
runner_agents	runner/AGENTS.md	document		runner working rules and local sandbox diagnostics		`Sandboxed automation harnesses`	`check_harness_guidance_routes`	
tests_readme	tests/README.md	document		the harness reference and suite table		`Suite coverage`	`check_index_vs_disk`	

Id	Node	Kind	Owns	Carries	Guards	Sources	Checks	Limits
tests_agents	tests/AGENTS.md	document		test working rules and local sandbox diagnostics		`Sandboxed automation harnesses`	`check_harness_guidance_routes`	
coverage_md	tests/COVERAGE.md	document		the outcome coverage matrices		`Normalized outcome coverage matrix`	`check_normalized_outcomes_have_matrix_rows`	
api_swift	PWRunnerAPI.swift	source		the Swift contract versions and the outcome vocabularies		`PWContract`; `NormalizedOutcome`	`check_normalized_outcomes_have_matrix_rows`	
request_swift	PWRunnerRequest.swift	source		the Swift request types and test overrides		`PWRunnerTestOverrides`	`check_test_overrides_table_agreement`	
json_contract_rs	json_contract.rs	source		the Rust contract versions; the envelope frame		`SCHEMA_VERSION`	`test_every_generated_copy_is_current`	
contract_py	tests/lib/contract.py	source		the Python contract versions and the worker identity		`CONTROLLER_ENVELOPE`; `WORKER_IDENTITY`	`test_every_generated_copy_is_current`; `test_identity_is_current_relocatable_and_regeneration_is_idempotent`	
abi_header	pw_probe_runner_abi.h	source		the C identity region and the shared-region layout		`PW_WORKER_ABI_IDENTITY_HEX`	`c_swift_layout_agreement`	
cworker_swift	CWorker.swift	source		the Swift identity region and the layout mirror		`abiIdentityHex`	`c_swift_layout_agreement`	
cli_rs	cli.rs	source		the usage text the controller README mirrors		`pub fn run`	`check_cli_surface_agreement`	
identity_sources	worker and runner sources	source		every byte the identity digest covers		`SOURCE_DIRS`	`test_new_protocol_helpers_are_discovered_but_tests_are_not_identity_inputs`; `test_native_manifest_inputs_change_identity`	
meson_build	meson.build	source		the native targets and their explicit source lists		`pwrunner_core`	`check_meson_source_lists`	
suites	tests/suites//	source		a README per suite and the case scripts the catalog names; no script is an entry point		`suite_directories`	`check_index_vs_disk`; `check_run_scripts_are_catalog_commands`	
drift_check	source_drift check.py	check			copies and vocabularies that are not generated, workflow guidance sections and diagnostic routes, the Known gap indexes in the architecture, build and worker tours, and the evidence-channel landing paths	`check_harness_guidance_routes`	`check_readmes_present`; `check_index_vs_disk`; `check_run_scripts_are_catalog_commands`; `check_no_direct_case_commands_in_docs`; `check_release_production_runs_the_default_battery`; `check_baseline_in_catalog_defaults`; `check_runner_outcome_suites_have_matrix_rows`; `check_normalized_outcomes_have_matrix_rows`; `check_attempt_outcomes_have_matrix_rows`; `check_attempt_kind_enum_agreement`; `check_prediction_unavailable_agreement`; `check_prediction_unavailable_planner`; `check_test_overrides_table_agreement`; `check_harness_guidance_routes`; `check_cli_surface_agreement`; `check_host_invariance`; `check_known_gap_index`; `check_evidence_channel_paths`; `check_meson_source_lists`; `runner_source_manifests_agree`	
drift_limits	source_drift limits.py	check			the limits generator's copies and the standalone guide's structure and internal links	`LimitsDocumentationTests`	`test_document_is_current`	
drift_contract	source_drift contract.py	check			the contract and identity generators' copies	`ContractVersionTests`; `WorkerIdentityTests`	`contract_versions`	

Id	Node	Kind	Owns	Carries	Guards	Sources	Checks	Limits
drift_architecture	source_drift architecture.py	check			this manifest against its dot, SVG and document copies	`ArchitectureDocumentationTests`	`architecture_documentation`	
gen_common	generator_common.py	generator	the citation forms, the duration pattern, the limits formatter and the span rules the three document generators share			`FORM_RULES`; `render_spans`	`test_check_citations_carry_a_form_and_tests_are_defined`; `test_readme_form_table_is_a_copy_of_the_shared_rules`	
prose_baseline	tests/fixtures/docs/prose_baseline.json	manifest	every prose site that still states a value, a count or a citation in an unverified form			`BASELINE_NAME`	`test_prose_baseline_is_consistent_and_growth_is_explicit`	
drift_generators	source_drift generators.py	check			the uniform generator invariants, selected prose links, the baseline's consistency, the drift node's rule list and the form table's copy	`GeneratorContractTests`; `broken_links`	`generator_contract`	
preflight	release_preflight.py	check			the baseline against the previous release tag	`inspect_baseline`	`test_release_preflight_refuses_a_grown_baseline`	
build	build.sh	check			checks the generated copies, selects the SDK, refuses a build directory configured for another checkout, compiles the native executables through Meson, checks the configured source lists against the tree and every output's minimum version, assembles and signs the bundle, checks every executable's signer and stages the guide	`generate_worker_identity.py`; `MESON_SOURCE_DIR`; `meson compile`; `native_sources.py`; `check_minimum_macos`; `signer_check.py`; `--stage-guide`	`test_document_is_current`; `problems`	
build_json	docs/build.json	manifest	the build's steps, refusals, knobs, signing list, helper invocations and directories, each with its citations			`MANIFEST_NAME`	`test_document_and_figure_are_current_and_the_rules_agree`	
gen_build	generate_build.py	generator	the build figure, its dot and SVG files, the generated build tables and the build spans; refuses a manifest that disagrees with build.sh			`parse_script`	`test_document_and_figure_are_current_and_the_rules_agree`	
build_md	docs/BUILD.md	document		the build figure and tables; local signing and compiler-discovery diagnostics		`REGION_START`	`test_document_and_figure_are_current_and_the_rules_agree`	
build_baseline	tests/fixtures/docs/build_baseline.json	manifest	every build refusal that no behavioral or helper control produces, with the control that would			`BASELINE_NAME`	`check_baseline`	
drift_build	source_drift build.py	check			the build manifest against build.sh, meson.build, the Makefile, the inventories and the baseline, and its copies	`BuildDocumentationTests`	`build_documentation`	
worker_md	docs/WORKER.md	document		resource inventory, capability figure and tables		`SPAN_DOCUMENTS`	`test_routing_and_second_document_refusals`	
markdown_reader	markdown_reader.py	library	the only import of the vendored markdown-it-py 3.0.0 and mdurl 0.1.2, and the Markdown structure the generators and checks read: blocks, headings and anchors, sections, links, inline code, list items and table cells			`PARSER`; `__version__ = "3.0.0"`	`markdown_reader`; `test_vendored_files_are_the_wheels_bytes`; `test_only_the_reader_imports_the_parser`	
extensions_md	docs/EXTENSIONS.md	document		the shared sandbox-extension coverage the guide copies		`BEGIN SHARED EXTENSION COVERAGE`	`test_extension_coverage_is_copied_from_its_document_demoted_and_standalone`	

The document graph edges

Id	From	To	Kind	Edge	Sources	Checks	Limits
D1	contract_json	gen_contract	reads	version numbers	`TARGETS`	`test_every_generated_copy_is_current`	

Id	From	To	Kind	Edge	Sources	Checks	Limits
D2	gen_contract	contract_md	writes	version sentence and table	`TARGETS`	`test_every_generated_copy_is_current`	
D3	gen_contract	api_swift	writes	PWContract	`TARGETS`	`test_every_generated_copy_is_current`	
D4	gen_contract	json_contract_rs	writes	SCHEMA_VERSION and the schema constants	`TARGETS`	`test_every_generated_copy_is_current`	
D5	gen_contract	contract_py	writes	version constants	`TARGETS`	`test_every_generated_copy_is_current`	
D6	gen_contract	guide	writes	version sentence	`TARGETS`	`test_every_generated_copy_is_current`	
D7	gen_contract	runner_readme	writes	version sentence	`TARGETS`	`test_every_generated_copy_is_current`	
D8	gen_contract	controller_readme	writes	version sentence	`TARGETS`	`test_every_generated_copy_is_current`	
D9	limits_json	gen_limits	reads	limits with citations	`load_limits`	`test_document_is_current`	
D10	matrix_json	gen_limits	reads	scenario rows	`MATRIX_NAME`	`test_document_is_current`	
D11	questions_md	gen_limits	reads	shared questions	`QUESTIONS_START`	`test_document_is_current`	
D12	failure_contract	gen_limits	reads	shared reading rules and extension statuses	`RULES_START`; `EXTENSION_START`	`test_document_is_current`; `test_extension_statuses_are_complete_and_copied_from_the_contract`	
D13	gen_limits	failure_contract	writes	scenario matrix	`MATRIX_START`	`test_document_is_current`	
D14	gen_limits	limits_md	writes	limits tables and coverage	`COVERAGE_START`	`test_document_is_current`	
D15	gen_limits	guide	writes	copied limits, questions, reading rules and extension statuses; staged copy at build	`GUIDE_START`; `--stage-guide`; `GUIDE_EXTENSION_START`	`test_document_is_current`	
D16	identity_sources	gen_identity	reads	sorted paths and contents, length-framed	`SOURCE_DIRS`	`test_new_protocol_helpers_are_discovered_but_tests_are_not_identity_inputs`; `test_native_manifest_inputs_change_identity`	
D17	gen_identity	abi_header	writes	identity bytes	`TARGETS`	`test_identity_is_current_relocatable_and_regeneration_is_idempotent`; `test_layout_and_both_handshake_implementations_change_identity`	
D18	gen_identity	cworker_swift	writes	identity bytes	`TARGETS`	`test_identity_is_current_relocatable_and_regeneration_is_idempotent`; `test_layout_and_both_handshake_implementations_change_identity`	
D19	gen_identity	contract_py	writes	identity hex	`TARGETS`	`test_identity_is_current_relocatable_and_regeneration_is_idempotent`; `test_layout_and_both_handshake_implementations_change_identity`	
D20	architecture_json	gen_architecture	reads	nodes, edges, styles and citations	`load_manifest`	`test_document_and_figures_are_current`	
D21	gen_architecture	architecture_md	writes	figure regions; dot and stamped SVG files beside the document	`expected_outputs`; `render_region`; `stamp_svg`	`test_document_and_figures_are_current`	
D22	agents_md	runner_agents	links	links to workflow-local sandbox diagnostics; destination section and root route are checked	`HARNESS_GUIDANCE_ROUTES`	`check_harness_guidance_routes`	
D23	agents_md	tests_agents	links	links to workflow-local sandbox diagnostics; destination section and root route are checked	`HARNESS_GUIDANCE_ROUTES`	`check_harness_guidance_routes`	
D24	cli_rs	controller_readme	copies	usage text	`check_cli_surface_agreement`	`check_cli_surface_agreement`	
D25	api_swift	coverage_md	copies	one matrix row per outcome constant	`check_normalized_outcomes_have_matrix_rows`	`check_normalized_outcomes_have_matrix_rows`	

Id	From	To	Kind	Edge	Sources	Checks	Limits
D26	request_swift	runner_readme	copies	test-seam table from PWRunnerTestOverrides	`check_test_overrides_table_agreement`	`check_test_overrides_table_agreement`	
D27	suites	tests_readme	copies	one suite-table row per suite	`check_index_vs_disk`	`check_index_vs_disk`	
D28	suites	catalog_json	copies	suite directories equal catalog suites	`check_index_vs_disk`	`check_index_vs_disk`	
D29	drift_check	agents_md	checks	nonempty sandbox guidance sections at workflow entry points and root links to their diagnostics	`check_harness_guidance_routes`	`runner_source_manifests_agree`	
D30	drift_check	controller_readme	checks	CLI surface block equals cli.rs usage	`check_cli_surface_agreement`	`runner_source_manifests_agree`	
D31	drift_check	coverage_md	checks	every outcome constant has a row and every row a constant	`check_normalized_outcomes_have_matrix_rows`	`runner_source_manifests_agree`	
D32	drift_check	tests_readme	checks	suite table equals suites and catalog	`check_index_vs_disk`	`runner_source_manifests_agree`	
D33	drift_limits	gen_limits	checks	--check, stale-copy and standalone-guide staging controls	`test_check_and_staging_refuse_stale_or_incomplete_documents_without_writing`	`limits_documentation`	
D34	drift_contract	gen_contract	checks	--check and generator controls	`ContractVersionTests`	`contract_versions`	
D35	drift_contract	gen_identity	checks	regeneration, relocation and stale-value controls	`WorkerIdentityTests`	`contract_versions`	
D36	drift_architecture	gen_architecture	checks	--check, citation, stale-copy and SVG-stamp controls	`ArchitectureDocumentationTests`	`architecture_documentation`	
D37	build	gen_limits	checks	--check before compiling; --stage-guide into dist	`--stage-guide`	`test_document_is_current`	
D38	build	gen_contract	checks	--check before compiling	`generate_contract.py`	`test_build_checks_every_generator_before_signing`	
D39	build	gen_identity	checks	--check before compiling and again before signing	`generate_worker_identity.py`	`test_build_checks_every_generator_before_signing`	
D40	build	gen_architecture	checks	--check before compiling	`generate_architecture.py`	`test_build_checks_every_generator_before_signing`	
D41	gen_common	gen_limits	reads	shared citation, formatter and span rules	`from generator_common import`	`test_limits_check_forms_and_value_owners`	
D42	gen_common	gen_architecture	reads	shared citation, placeholder and span rules	`from generator_common import`	`test_check_citations_carry_a_form_and_tests_are_defined`	
D43	limits_json	gen_architecture	reads	limit placeholders in facts, labels and notes, and the Limits column	`resolve_graph`	`test_durations_and_sizes_render_from_limits`	
D44	drift_generators	gen_contract	checks	nothing changes outside its regions; --check before signing	`test_regeneration_changes_nothing_outside_regions`	`test_regeneration_changes_nothing_outside_regions`	
D45	drift_generators	gen_identity	checks	nothing changes outside its regions; --check before signing	`test_regeneration_changes_nothing_outside_regions`	`test_regeneration_changes_nothing_outside_regions`	
D46	drift_generators	gen_limits	checks	ownership, spans, forms and value owners, and whole-file staging	`test_whole_file_staging_is_owned_and_idempotent`	`test_spans_are_authored_outside_regions_and_copied_as_bytes`	
D47	drift_generators	gen_architecture	checks	ownership, spans, placeholders, declared columns, captions and render refusal	`test_architecture_render_failure_leaves_every_copy_untouched`	`test_captions_state_the_verified_guarantee`	
D48	drift_generators	prose_baseline	checks	the listed sites equal the scanned sites; growth only by an explicit entry	`baseline_problems`	`test_prose_baseline_is_consistent_and_growth_is_explicit`	
D49	preflight	prose_baseline	checks	a release's baseline is a subset of the previous release's	`inspect_baseline`	`test_release_preflight_refuses_a_grown_baseline`	
D50	drift_check	architecture_md	checks	Known gap paragraphs equal the index; evidence-channel paths resolve in the shape goldens	`check_known_gap_index`; `check_evidence_channel_paths`	`runner_source_manifests_agree`	
D51	drift_generators	architecture_json	checks	the drift node's rule citations equal the rules check.py runs	`test_document_graph_cites_every_drift_rule`	`test_document_graph_cites_every_drift_rule`	
D52	drift_check	meson_build	checks	source list equals the tree	`check_meson_source_lists`	`runner_source_manifests_agree`	
D53	build_json	gen_build	reads	steps, refusals, knobs, signing, invocations, directories and citations	`load_manifest`	`test_document_and_figure_are_current_and_the_rules_agree`	

Id	From	To	Kind	Edge	Sources	Checks	Limits
D54	build	gen_build	reads	banners, refusals, signing calls, knobs and helper invocations, parsed	`parse_script`	`check_banner_order`; `test_script_mutations_are_named_by_their_rule`	
D55	gen_build	build_md	writes	figure, step, refusal, signing, knob and directory regions; dot and stamped SVG beside the document	`render_document`; `stamp_svg`	`test_document_and_figure_are_current_and_the_rules_agree`	
D56	build	gen_build	checks	--check before compiling	`generate_build.py`	`test_build_checks_every_generator_before_signing`	
D57	drift_build	gen_build	checks	--check, the ten grounding rules, parser refusals, citation, stale-copy and mutation controls	`BuildDocumentationTests`	`build_documentation`	
D58	drift_build	build_baseline	checks	the listed refusals equal the uncovered ones; growth only by an explicit entry, refused at release	`baseline_problems`	`test_release_preflight_refuses_a_grown_build_baseline`	
D59	gen_common	gen_build	reads	shared citation, placeholder and span rules; the architecture generator's table and stamp rendering	`from generator_common import`; `import generate_architecture as arch`	`test_document_and_figure_are_current_and_the_rules_agree`	
D60	drift_generators	gen_build	checks	nothing changes outside its regions; --check before signing; one span owner	`test_regeneration_changes_nothing_outside_regions`	`test_regeneration_changes_nothing_outside_regions`	
D61	drift_check	build_md	checks	Known gap paragraphs equal the index, as in the architecture document	`known_gap_index_problems`	`runner_source_manifests_agree`	
D62	agents_md	build_md	links	links to workflow-local sandbox diagnostics; destination section and root route are checked	`HARNESS_GUIDANCE_ROUTES`	`check_harness_guidance_routes`	
D65	gen_architecture	worker_md	writes	per-graph destination; capability figure and inventory beside WORKER.md	`graph_document`	`test_routing_and_second_document_refusals`	
D66	drift_check	worker_md	checks	Known gap paragraphs equal the final index in document order	`check_known_gap_index`	`runner_source_manifests_agree`	
D67	markdown_reader	gen_common	reads	Markdown citations and span placement	`import markdown_reader`	`test_markdown_citations_name_headings_heading_code_or_markers`; `test_a_span_marker_that_begins_a_line_is_refused`	
D68	markdown_reader	gen_limits	reads	the guide's headings, rows, lists and links and the standalone refusals	`import markdown_reader`	`test_constructs_in_comments_or_code_satisfy_no_structural_check`	
D69	markdown_reader	gen_architecture	reads	heading anchors for figure and limit links	`markdown_reader.slug`	`test_anchors_keep_underscores_and_number_duplicates`	
D70	markdown_reader	gen_build	reads	README.md's bundle inventory and figure anchors	`import markdown_reader`	`test_figure_labels_links_and_paths_render_as_claimed`; `test_meson_makefile_inventory_and_baseline_mutations_are_named`	
D71	markdown_reader	drift_check	reads	sections, tables, lists, links and guidance routes	`import markdown_reader`	`runner_source_manifests_agree`	
D72	markdown_reader	drift_generators	reads	G9 prose lines, citation pairs and G11 links	`import markdown_reader`	`test_prose_links_resolve_anchors_and_symbol_links`; `test_prose_baseline_is_consistent_and_growth_is_explicit`	
D63	extensions_md	gen_limits	reads	shared extension coverage	`COVERAGE_SHARED_START`	`test_extension_coverage_copy_rejects_staleness_dependencies_and_missing_regions_without_writes`	

Every node and edge above cites at least one test or rule.

BYOXPC as a variation on launch and selection

A BYOXPC runner is a copy of the XPC bundle registered as a launchd Mach service, in user or system scope, by the `runner` subcommands (controller/README.md). The client connects with `NSXPCConnection(machServiceName:)`. The host binds `NSXPCListener(machServiceName:)` and resolves its worker, validator and issuer relative to its own bundle, exactly as the built-in one does. The request names the runner through the selector fields the controller strips before XPC delivery, and the registry supplies the service name and scope. The installer signs the copy's embedded worker with the supplied identity and entitlements

plist, the validator and issuer with the identity alone, then the enclosing bundle, verifies the seal recursively, and the registry records each binary's signature and entitlements as read back (`sign_install_tree`, `read_back` in runner_manager.rs). The dossier records the hashes of the bundle's service, worker, validator and issuer files as observed before invocation and compares them with the manifest baselines (`byoxpc_run_with_a_manifest_compares_the_bundle_copies_with_its_baselines` in run_flow.rs); the guide defines how to interpret these pre-invocation observations. The registry is read by an additive loader that still accepts retired kind spellings so an old install can be listed and removed; recovery of a half-removed install is the registry ownership procedure. Absent issuer fields in older registry records remain unknown; validation and removal still work. A current runner copy without the issuer can serve ordinary and actor-form specimens but receives a named capability refusal for declaration-form extension specimens. Runtime dependency is separate from the complete helper inventory required for a new installation. Shared standard/BYOXPC extension and phase controls establish the behavior with independent binary read-back and verified owned cleanup.

`--env` configures the host's launchd environment. The worker's environment is not recorded in the reply; actor launches explicitly supply an empty environment, and the single-worker launch account is in WORKER. Each worker spawns exec children with an empty environment.

The apparatus itself, from the copied bundle to the processes the policy reaches, has the last figure. Two of its facts are easy to miss from the guide's install recipe: `runner verify` sends a real request, so it consumes the host it reaches, and the next request meets that host's terminal claim or encounters launchd pacing configured by the plist's `ThrottleInterval` of 1 second, which does not guarantee a request-delivery time; and a selector's `required_entitlements` is satisfied only by keys the worker's read-back records with the value `true`.

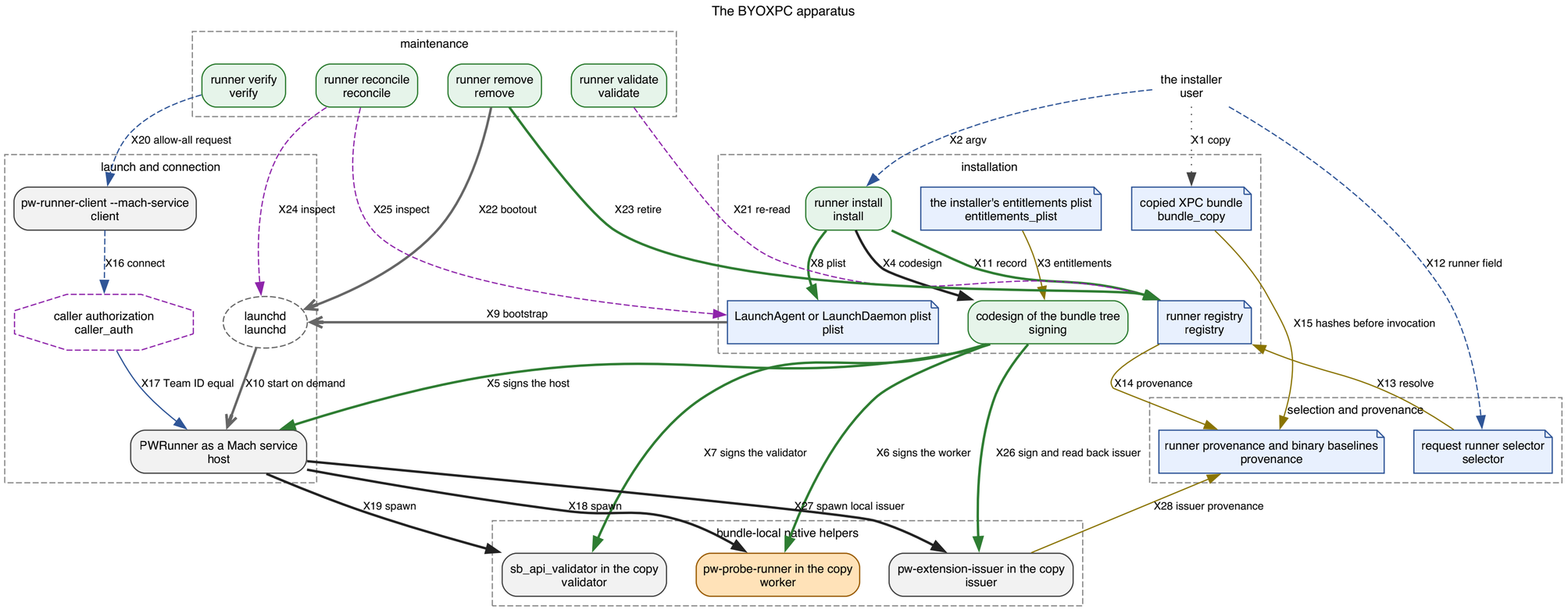


Figure: the byoxpc apparatus. Generated from architecture.json by generate_architecture.py; dot source in architecture-byoxpc.dot. The ids in the figure are the ids in the tables below, and each row cites the source symbol that implements it and the test or rule the manifest names for it. Symbol presence and test definition are verified; whether a test asserts the row is not verified.

The BYOXPC apparatus: 20 nodes and 28 edges, with symbol presence and test definition verified; whether a test asserts the row is not verified

The BYOXPC apparatus nodes

Id	Node	Kind	Role	Contents	Ownership	Behavior	Identity	Checks	Location	Lifecycle	Sources	Checks	Limits
user	the installer	person	a person or a harness	a bundle copy, an identity, an entitlements plist, a scope							`Install a BYOXPC runner`	`install`; `runner_install`	
bundle_copy	copied XPC bundle	record		the host, worker, validator and issuer, each with the build's signature until the installer re-signs them; CFBundleIdentifier is the service name	the installer						`Install a BYOXPC runner`; `sign_macho`	`owned_bundle`; `runner_install`	
entitlements_plist	the installer's entitlements plist	record			the installer	the host executable and the embedded worker, the process the policy is applied to					`--entitlements`	`binary_entitlements`; `runner_install`	
install	runner install	command				signs the worker, validator, issuer and then the bundle, verifies the seal recursively, writes the launchd plist, bootstraps, records the runner with every binary's read-back; with --env, adds EnvironmentVariables					`cmd_runner_install`; `--env`; `sign_install_tree`	`runner_install`; `install`; `entitlement_readback`	
signing	codesign of the bundle tree	command					the embedded worker with the identity and the supplied entitlements, the validator and issuer with the identity alone, then the enclosing bundle with the identity and the entitlements	the sealed bundle recursively (--deep --strict) before anything is recorded			`sign_install_tree`; `codesign_sign`; `codesign_verify`	`install_signing_signs_helpers_before_the_bundle`; `byoxpc_verification_controls`; `binary_entitlements`	
plist	LaunchAgent or LaunchDaemon plist	record		MachServices, RunAtLoad, ProgramArguments with --mach-service, EnvironmentVariables from --env, ThrottleInterval of 1 second					the user's LaunchAgents or / Library/ LaunchDaemons		`build_launchd_plist`; `MachServices`; `EnvironmentVariables`; `launchd_plist_path`; `BYOXPC_THROTTLE_INTERVAL_SECONDS`	`byoxpc_setup`; `runner_install`; `byoxpc_plist_sets_the_respawn_throttle`	`byoxpc_throttle_interval`
registry	runner registry	record		service name, scope, bundle path, and separate host/worker/ validator/issuer signature and entitlement read-backs, each entitlement set with its keys and the keys granted (value true); absent helper observations in older records remain unknown						pending, installed, pending_cleanup	`install_record`; `RunnerState`; `PendingCleanup`; `entitlements_from_codesign`; `read_back`	`legacy_machme_kind_deserializes_as_byoxpc`; `registry_recovery`; `registry_records_without_helper_read_backs_still_load`; `registry_records_without_helper_read_backs_still_load`	
launchd	launchd	os				the host on demand for its Mach service				one launch per ThrottleInterval, 1 second in the generated plist; launchd's own default applies without the key	`launchctl_bootstrap`; `launchctl_target`	`pwrunner_survives_mach_service_launch`; `service_present`	`byoxpc_throttle_interval`
host	PWRunner as a Mach service	executable				NSXPCListener(machServiceName:) from --mach-service	the installed identity and entitlements		relative to its own bundle		`NSXPCListener(machServiceName:`; `defaultWorkerExecutablePath`	`pwrunner_survives_mach_service_launch`; `single_use`	

Id	Node	Kind	Role	Contents	Ownership	Behavior	Identity	Checks	Location	Lifecycle	Sources	Checks	Limits
client	pw-runner-client --mach-service	executable				NSXPCConnection(machServiceName:), with the privileged option for system scope					`--mach-service`; `--privileged`	`runner_install`; `runClientSessionControls`	
caller_auth	caller authorization	check		PWRunnerRequireSignedCaller and PWRunnerAllowedIdentifiers in the host's Info.plist, inherited by a copy				the caller's Team ID equals the host's; an optional identifier allowlist			`authorizedCaller`; `PWRunnerRequireSignedCaller`; `PWRunnerAllowedIdentifiers`	`runner_auth_external`	
selector	request runner selector	record		the runner by id or service, with mode and required_entitlements				every required key present with the value true in the worker's recorded read-back; the host's entitlements are recorded, not consulted; a record without a worker read-back is refused with a reinstall message when anything is required		before XPC delivery	`parse_runner_selector_value`; `enforce_required_entitlements`; `strip_runner_selector`	`malformed_selection_never_falls_back_or_discards_entitlements`; `selection_refuses_a_false_valued_required_key`; `entitlement_gate_consults_the_worker_not_the_host`	
provenance	runner provenance and binary baselines	record		runner kind, service name, registry id and separate host/worker/validator/issuer signature and entitlement read-backs; selected binary hashes observed before invocation; missing older issuer observations remain unknown							`runner_provenance_from_target`; `observe`	`byoxpc_run_with_a_manifest_compares_the_bundle_copies_with_its_baselines`; `byoxpc_binaries`; `dossier_witness_byoxpc`; `extensions_byoxpc`	
worker	pw-probe-runner in the copy	sandboxed				the installed plist's entitlements; a require-entitlement rule consults this process	the installed identity and the supplied entitlements, signed before the bundle seal				`sign_install_tree`; `defaultWorkerExecutablePath`	`single_use`; `entitlement_transfer`	
validator	sb_api_validator in the copy	executable					the installed identity alone, no entitlements				`sign_install_tree`; `defaultValidatorExecutablePath`	`runner_install`; `entitlement_readback`	
verify	runner verify	command				sends a fixed allow-all request through the client and waits 5,000 milliseconds by default				one reserved host and at most one execution; subsequent acquisition can wait through occupied hosts or the generated plist throttle, without sending a specimen before its own grant	`cmd_runner_verify`; `verify_request`; `PWRunnerAdmission`	`verification_request_is_the_fixed_allow_all_specimen`	`runner_verify_wait`
validate	runner validate	command				verifies each record's bundle recursively and the host, worker and validator individually; also verifies the issuer when its file exists or prior issuer evidence is recorded; re-reads those signatures and entitlement sets, reports failures by record and binary, and leaves an absent unrecorded issuer unknown; consults no launchctl					`cmd_runner_validate`; `codesign_metadata`; `validate_record`	`validation_reports_the_bundle_and_each_binary_that_fails`; `byoxpc_verification_controls`; `runner_install`	

Id	Node	Kind	Role	Contents	Ownership	Behavior	Identity	Checks	Location	Lifecycle	Sources	Checks	Limits
remove	runner remove	command				moves ownership to pending_cleanup, boots the service out, removes the plist, retires the record after verified absence					`cmd_runner_remove`; `bootout_service`; `cleanup_complete`	`registry_recovery`; `cleanup`	
reconcile	runner reconcile	command				reports launchd services and plists that look owned, changing nothing					`cmd_runner_reconcile`; `inspect_service`; `inspect_plist`	`runner_install`	
issuer	pw-extension-issuer in the copy	executable				bundle-local ordinary-authority issuance before the worker	installed identity alone, no supplied worker entitlements				`sign_install_tree`; `defaultIssuerExecutablePath`	`extensions_byoxpc`; `byoxpc_verification_controls`	

The BYOXPC apparatus edges

Id	From	To	Kind	Edge	Sources	Checks	Limits
X1	user	bundle_copy	copies	copies the shipped PWRunner.xpc	`Install a BYOXPC runner`	`owned_bundle`; `runner_install`	
X2	user	install	channel	runner install with --bundle, --identity, --entitlements, --scope and optional --env	`cmd_runner_install`	`install`; `runner_install`	
X3	entitlements_plist	signing	reads	supplied entitlements	`--entitlements`	`entitlements`; `runner_install`	
X4	install	signing	spawn	sign_install_tree: codesign the worker with --entitlements, the validator, the issuer, then the bundle; codesign --verify --deep --strict	`sign_install_tree`; `codesign_verify`	`entitlements`; `runner_install`	
X5	signing	host	writes	the host executable carries the identity and entitlements; read back by entitlements_from_codesign and codesign_metadata	`entitlements_from_codesign`	`entitlements`; `runner_install`	
X6	signing	worker	writes	the worker carries the identity and the supplied entitlements; read back by read_back	`sign_install_tree`; `read_back`	`entitlement_readback`	
X7	signing	validator	writes	the validator carries the identity alone; read back by read_back	`sign_install_tree`; `read_back`	`entitlement_readback`	
X8	install	plist	writes	build_launchd_plist and write_launchd_plist	`build_launchd_plist`; `write_launchd_plist`	`byoxpc_setup`	
X9	plist	launchd	launch	launchctl bootstrap into the gui or system domain	`launchctl_bootstrap`; `launchctl_target`	`service_present`; `runner_install`	
X10	launchd	host	launch	start on demand for the Mach service; configured ThrottleInterval 1 second paces launches without guaranteeing request-delivery time	`MachServices`; `NSXPCListener(machServiceName:`	`pwrunner_survives_mach_service_launch`	`byoxpc_throttle_interval`
X11	install	registry	writes	install_record: pending before the plist, installed after the bootstrap	`install_record`	`registry_recovery`	
X12	user	selector	channel	the specimen's runner field: mode, id or service, required_entitlements	`parse_runner_selector_value`	`parses_runner_selector_from_nested_runner`	
X13	selector	registry	reads	resolve by id or service; every required entitlement must be granted (value true) in the worker's recorded read-back	`resolve_runner_target_with_registry`; `enforce_required_entitlements`	`malformed_selection_never_falls_back_or_discards_entitlements`; `selection_refuses_a_false_valued_required_key`	
X14	registry	provenance	reads	runner_provenance_from_target copies kind, service, id and each binary's signature and entitlements	`runner_provenance_from_target`	`byoxpc_binaries`; `extensions_byoxpc`	
X15	bundle_copy	provenance	reads	Binaries::observe hashes the service, worker, validator and issuer files before invocation	`observe`	`byoxpc_run_with_a_manifest_compares_the_bundle_copies_with_its_baselines`	
X16	client	caller_auth	channel	NSXPCConnection(machServiceName:), privileged for system scope	`--mach-service`	`runner_install`	

Id	From	To	Kind	Edge	Sources	Checks	Limits
X17	caller_auth	host	guard	shouldAcceptNewConnection: the caller's Team ID equals the host's, and the identifier allowlist if present	`shouldAcceptNewConnection`; `authorizedCaller`	`runner_auth_external`	
X18	host	worker	spawn	posix_spawn of the copy's own worker	`defaultWorkerExecutablePath`	`single_use`	
X19	host	validator	spawn	eligible collection spawns the copy's bundle-local validator by default; an explicit override takes precedence	`defaultValidatorExecutablePath`; `runReservedSpecimen`	`runner_install`; `runValidatorSelectionTests`	
X20	verify	client	channel	a fixed allow-all request on the client's stdin	`verify_request`	`verification_request_is_the_fixed_allow_all_specimen`	
X21	validate	registry	checks	verifies the bundle recursively and each binary on its own, re-reads every signature and entitlement set, reports failures	`cmd_runner_validate`; `validate_record`	`byoxpc_verification_controls`	
X22	remove	launchd	launch	bootout, then re-read for up to 1,000 milliseconds of teardown	`bootout_service`	`cleanup`; `runner_install`	`runner_remove_teardown_wait`
X23	remove	registry	writes	pending_cleanup first; retired only after verified absence of service and plist	`PendingCleanup`; `cleanup_complete`	`registry_recovery`	
X24	reconcile	launchd	checks	inspect_service lists candidate services without changes	`inspect_service`	`runner_install`	
X25	reconcile	plist	checks	inspect_plist lists candidate plists without changes	`inspect_plist`	`runner_install`	
X26	signing	issuer	writes	issuer receives identity alone; signature and entitlements read back independently	`sign_install_tree`; `read_back`	`extensions_byoxpc`; `install_signing_signs_helpers_before_the_bundle`	
X27	host	issuer	spawn	declaration form uses copy-local issuer path; named declaration capability refusal if missing; actor issue runs in its own worker	`defaultIssuerExecutablePath`; `capabilityUnavailable`	`extensions_byoxpc`	
X28	issuer	provenance	reads	actual selected path and hash compared with shipped issuer baseline; separate installed read-backs	`issuer`; `runner_issuer_signature`	`extensions_byoxpc`	

Notes:

- `host`: All authorized connections share one locked reservation slot. Only the connection holding its grant can consume it, before specimen decoding. Stale expiry or disconnect cannot retire an executing owner; client loss after submission never permits replay.

Every node and edge above cites at least one test or rule.

Known gaps

Each gap is stated in full where the promise it limits is stated; this list only points there, in document order.

- Why these processes exist: a completed specimen can lose its reply, with no result retrieval or safe execution replay.